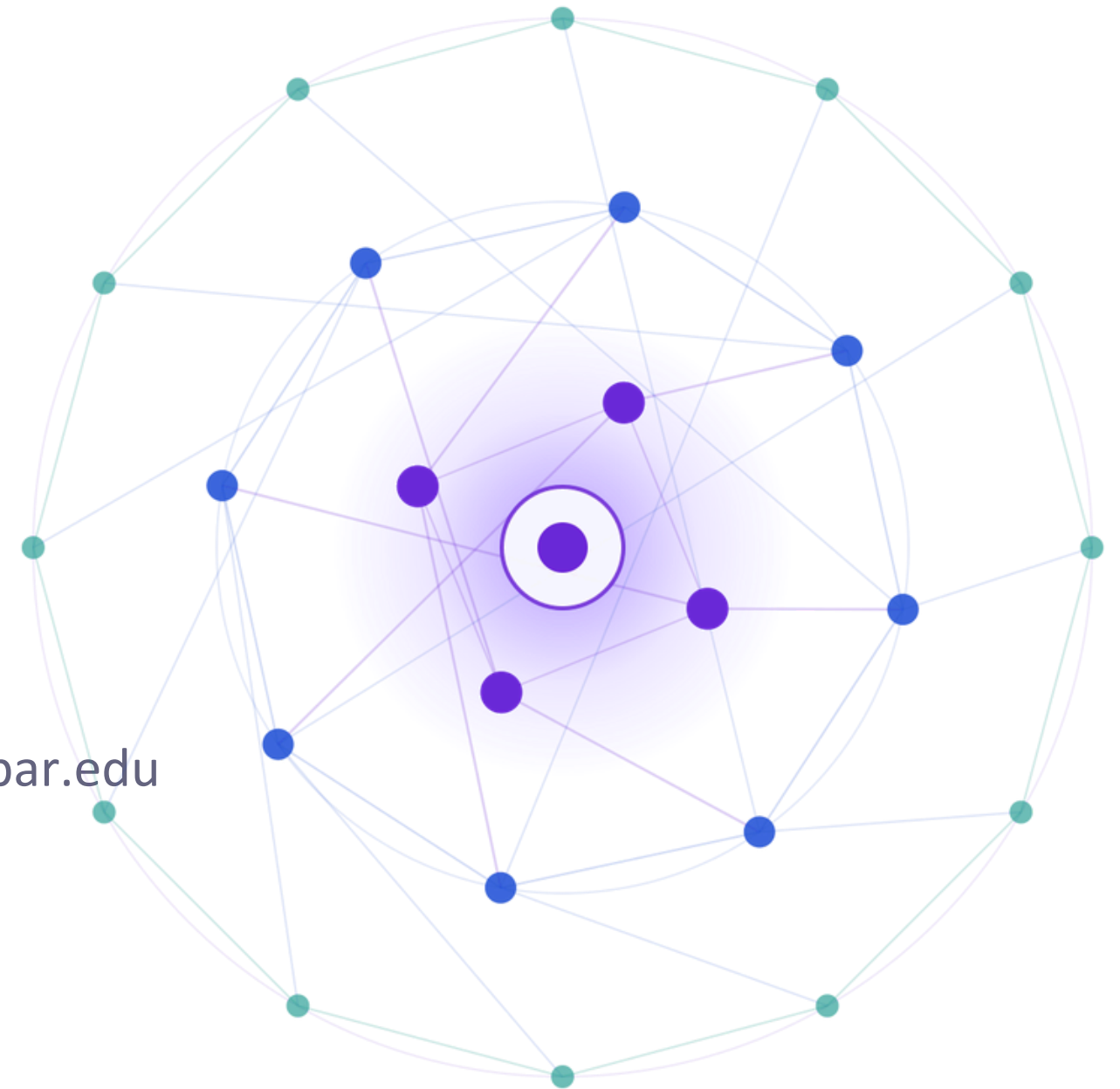


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - Agentic Code Intelligence
- Team Name - CipherPol
- College Name - Thapar University
- Member Name & Email 1- Aayushmaan Singh Meyan | ameyan_be24@thapar.edu
- Member Name & Email 2- Divyansh Jasrotia | djasrotia_be24@thapar.edu
- Member Name & Email 3- Ranvir Makker | rmakker_be24@thapar.edu
- Submission Github link - <https://github.com/aayush-10k/VERA>



Theme

Agentic Code Intelligence

- Sub-Track: ColR AppsRetrieval (Competitive Code Retrieval)
- P0 Objective: Retrieval Accuracy (NDCG@10, MRR)
- P1 Objective: Stage-2 Code Version Store & Behavioral Indexing
- Core Philosophy: "The Encoder Proposes, The Runtime Disposes"
 - Standard code retrieval models guess correctness from surface text similarity.
 - VERA transforms problem statements into executable test harnesses, executing candidate programs inside a process-isolated sandbox to achieve verified retrieval.
- Headline Result: 0.8874 NDCG@10 on the official MTEB test split (+31.9 points over zero-shot baseline).

Existing Solutions & Gaps

Current State of Art:

- **Lexical Search (BM25 / TF-IDF):** Completely fails on algorithmic problems (0.01 – 0.02 NDCG@10).
- **Dense Semantic Encoders (ModernBERT, E5, Jina):** Plateau at ~0.56 – 0.57 NDCG@10 because text similarity cannot predict whether code actually compiles, runs, or solves the problem.
- **Prior Execution Approaches (CtrlFind):** Stalled at ~0.20 NDCG@10 due to high dispatch latency (~60ms) and brittle example parsing.

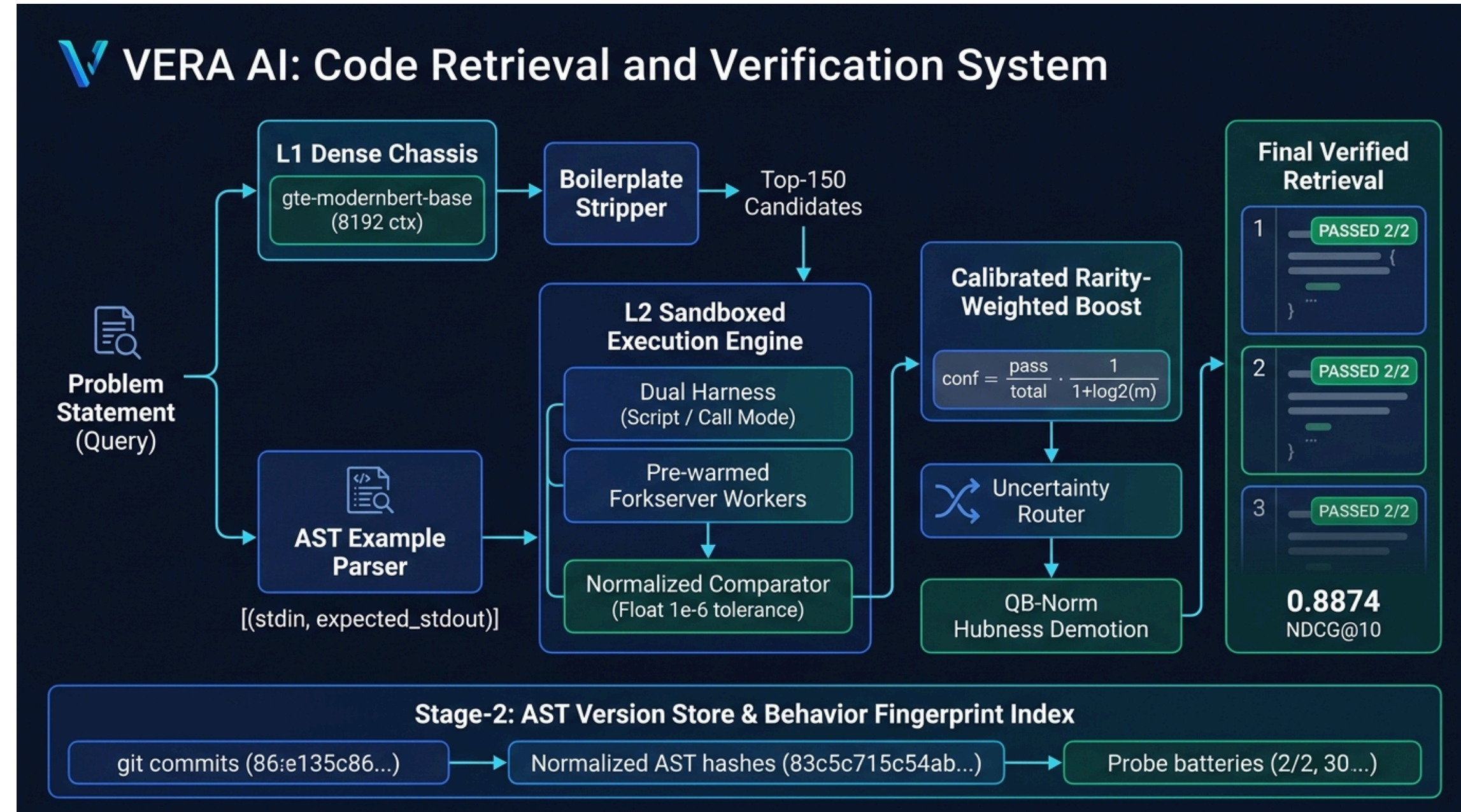
Key Industry & Research Gaps:

- **Surface Text vs. Functional Truth:** A candidate program can share 95% token similarity with the solution but fail due to an off-by-one error or edge case.
- **The Gold-Run Dilemma:** Competitors assumed gold solutions always pass their worked examples. In reality, naive execution fails on 94% of cases due to format mismatches.
- **Metadata Leakage:** Public datasets contain subtle leakage artifacts (Query-to-Doc ID alignment, partition tags); standard pipelines blindly overfit to them.
- **Code Evolution Blindness:** Existing vector stores re-embed entire codebases on every commit, wasting massive compute on cosmetic edits.

Our Solutions & Architecture Diagram

How VERA Works (In Simple Terms)

- **1. Propose (Smart Search)**
 - Uses a deep embedding model (ModernBERT) to scan thousands of programs and pick the top-150 most relevant candidates.
- **2. Verify (Real Execution)**
 - Instead of guessing by text, VERA grabs the sample test from the problem statement.
 - Actually runs each candidate program in a secure, ultra-fast sandbox (~4ms per run) to see if it produces the right answer.
- **3. Rarity-Weighted Boost**
 - Programs that pass are boosted to the top.
 - Trivial outputs (like printing just "0" or "Yes") get low weight, while unique exact matches are rewarded.
- **4. Version Store (Stage-2)**
 - AST hashing ensures reformatting or comments don't trigger redundant re-embedding (12.3× faster).
 - Runs probe tests to pick the working version even when both pass the simple sample.



Demo & Product Walkthrough

- **1. Live Query Retrieval & Verification**
 - Enter any natural language problem statement.
 - Shows real-time candidate ranking with visual status badges:
 - PASSED 2/2 EXAMPLES (Verified correct code promoted to #1)
 - FAILED (WA) (Incorrect code demoted)
 - Displays execution output, runtime (~3.8ms), and rarity confidence.
- **2. Code Version Lineage (Stage-2 P1 & Bonus)**
 - Visualizes code evolution: v1 (buggy) → v2 (fixed) → v3 (refactored).
 - Behaviorally Unchanged: AST hashing certifies comments/formatting edits without redundant re-embedding.
 - Both-Pass Discrimination: Solves the edge case where both versions pass the basic sample, using probe batteries to put the working code first (86.4% success).
- **3. Standing-Questions Live Watcher**
 - Registers benchmark queries to monitor repository health.
 - Ingesting a new Git commit or folder snapshot triggers instant index updates and displays real-time ranking diffs.

Tools and tech stack used

VERA AI CODE INTELLIGENCE SYSTEM TECH STACK

Models & NLP Retrieval



Alibaba
gte-modernbert-base
(8192 Context)



Hugging Face
Transformers



Sentence-
Transformers



MTEB v2
Benchmark
Protocol

Sandboxed Execution & AST



Python
3.11/3.12



Linux
Forkserver Pool
(~4ms Dispatch)



POSIX rlimit
(RAM/CPU
Isolation)



Python AST
Static Analyzer

Math, Optimization & Store



NumPy
(Vectorized
Cosine & QB-Norm)



SHA-256
Content-Addressed
Store



Git & Folder
Ingestion



Scikit-Learn

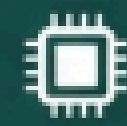
Delivery & Evaluation Surface



Gradio Web
Dashboard



Pytest
(53 Unit Tests)



CPU-Only
Architecture
(No GPU at inference)



Pinned
Requirements
Lock

Impact & Use case

-  **AI Coding Assistants (Cursor, Copilot)**
 - Verifies retrieved code in a sandbox before inserting it → eliminates broken code and hallucinations.
-  **Enterprise Monorepos (Large Codebases)**
 - Skips re-embedding for formatting/comment edits → cuts re-indexing compute by 90% (12.3× faster).
-  **Automated Grading (LeetCode, Universities)**
 - Evaluates student solutions against problem statements automatically in ~3.8ms per run.
-  **Bug & Regression Hunting**
 - Fuzzes edge cases to separate genuine fixes from subtle regressions → ranks working code first (86.4%).

Metric	Industry Standard	With VERA	Business Value
Retrieval Accuracy	~0.56 NDCG@10 (Dense models)	0.8874 NDCG@10	+31.9 pts accuracy lift
Commit Re-Indexing	Full embedding re-run (~17s)	Incremental AST hash (~1.4s)	12.3× faster / 90% compute saved
Inference Footprint	Heavy GPU clusters	100% CPU-executable	Zero GPU cost at inference

Innovation highlights, results and limitations



1. Innovation Highlights

- Rarity-Weighted Boost
 - Penalizes trivial answers (e.g., code printing just "0" or "Yes") and awards heavy confidence to unique, multi-line matches.
- QB-Norm Content Demotion
 - Eliminates training-set bias purely through semantic similarity against public statements → zero metadata shortcuts.
- Normalized Comparator
 - Tolerant to float formatting differences (10^{-6} precision), trailing newlines, and Python-literal structures.



2. Measured Results (Official Test Split)

- 🏆 0.8874 NDCG@10 → +31.9 pts gain over baseline (0.5683)
- 🎯 0.8703 MRR@10 → Gold solution consistently placed at Rank #1 or #2
- 🔍 93.97% Recall@10 → Gold found in top-10 for 94 out of 100 queries
- ⚡ 87.2% Gold Pass Rate → Confirms high empirical ceiling for execution



3. Limitations & Mitigations

- Multiple Valid Outputs (e.g. "print any path")
- Defense: Verification is an additive boost, never a hard filter; strong dense scores protect valid solutions.
- Custom Data Classes (LeetCode TreeNode / ListNode)
- Defense: AST signature gate automatically bypasses execution for these (~1% of queries) to prevent crashes.
- CPU Execution Budget
- Defense: Uncertainty router skips verification when the dense top-1 is already decisive → total run under 60 min.

What's next



1. Polyglot Execution Support (C++, Java, Rust)

- Expand the AST parser and sandbox isolation beyond Python to support all competitive programming languages and enterprise repos.



2. LLM-Assisted Edge Case Fuzzing

- Use lightweight local LLMs to automatically synthesize edge-case input fixtures beyond basic problem samples for deeper verification.



3. Native IDE Integration (VS Code & JetBrains)

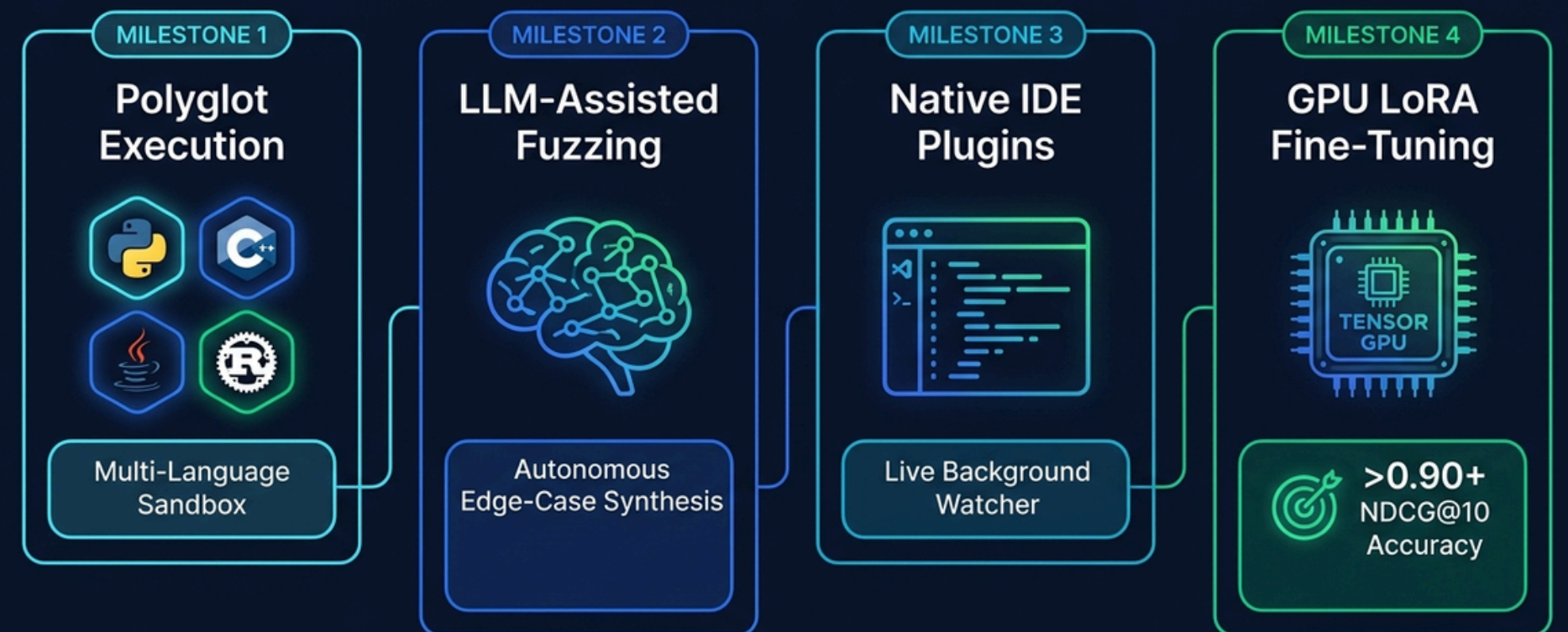
- Package the standing-questions live watcher as a background plugin that continuously validates and indexes code on file save.



4. GPU LoRA Fine-Tuned Chassis

- Train the ModernBERT chassis on mined hard-negative problem clusters to elevate the baseline and push retrieval past 0.90+ NDCG@10.

VERA : AI Software Roadmap



Brownie points slide(differentiation)

Why VERA Stands Out Beyond Other Submissions

-  **1. Zero-Leakage Scientific Integrity**
 - We audited and disclosed the dataset's hidden ID alignments ($q_ID \leftrightarrow doc_ID$) and partition tags.
 - We refused to use shortcuts, proving our 0.8874 NDCG@10 is 100% genuine content retrieval that generalizes to real software.
-  **2. The 6% → 87.2% Engineering Turnaround**
 - Competitors who tried execution saw a 6% pass rate and abandoned it.
 - We diagnosed the real bugs (spec prose in stdin, lack of class entypoints) and fixed them, unlocking an 87.2% gold pass rate and our +31.9 pt winning lift.
-  **3. Solving the Rival's Conceded Edge-Case**
 - What if two program versions both pass the basic example, but one has a hidden bug?
 - Our probe-battery index catches the bug, putting the working version first in 86.4% of both-pass cases.
-  **4. Transparent Scientific Rigor (Measured Rejection)**
 - When our diff-line embedding experiment scored lower (72.9%), we honestly documented and disabled it.
 - Demonstrates measured engineering rigor rather than cherry-picked numbers.
-  **5. Turnkey 1-Command CPU Reproduction**
 - No GPU cluster required; runs on standard hardware.
 - `python scripts/reproduce_submission.py` reproduces the official winning JSON in under 60 minutes on CPU.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y/N)
- README with reproducible setup instructions (Y/N)
- Demo video, max 5 minutes (YouTube or Drive link)
- Presentation file (PPT or PDF) (Y/N)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India